Deep Learning (Homework 2)

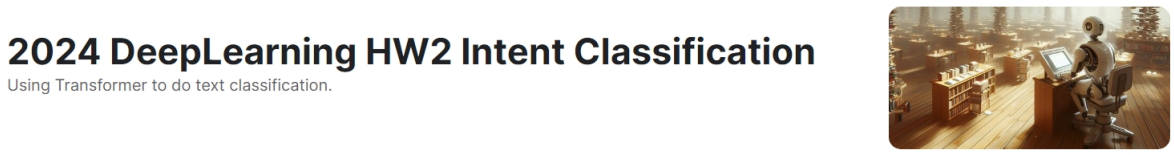
Due date : 2024/5/26 23:55:00 (Hard Deadline)

1 Intent Classification (50%)

To start this problem, some preliminary steps need to be conducted first:

1. **Join the in-class competition on Kaggle (Here)**
   (or search 2024 DeepLearning HW2 Intent Classification in Kaggle)
2. **Download the data and check for the description**
3. **You may allow changing your team name to student ID after the first submission.**

In this problem, you are given a CSV file, train.csv, containing more than 10,000 records of banking-related data. Each record consists of two columns: Text, containing the text of a customer inquiry, and Category, indicating the type of banking request (e.g., "account_balance", "card_declined). Your task is to build a Transformer to classify these inquiries into their corresponding categories and submit the classification result to the Kaggle competition.



1.1 Data Preprocessing (10%)

In this homework, you cannot directly use high-level API to help you process the text data. You are asked to convert a text string into a list of integers by these packages: FastText, TorchText, NLTK, spaCy or Gensim.

1. How do you choose the tokenizer for this task? Could we use the white space to tokenize the text? What about using the complicated tokenizer instead? Make some discussion. (5%) (You might want to explain it by showing the performance comparison with different tokenizer. If you are not familiar with tokenizer, check (`https://www.analyticsvidhya.com/blog/2019/07/how-get-started-nlp-6-unique-ways-perform-tokenization/`))
2. Why we need the special tokens like ⟨pad⟩, ⟨unk⟩? (2%)
3. Briefly explain how your procedure is run to handle the text data. (3%) (e.g. Which tokenizer do you choose? Why? What is your min_count setting? etc.)

1.2 Transformer (40%)

Build the Transformer (using high-level API is forbidden) to solve this task and answer the following questions. (Hint: You might want to read this tutorial first (`https://pytorch.org/tutorials/beginner/transformer_tutorial.html`))

1. Pass the Baseline on the Kaggle in-class competition (Public). (15%)

2. Discuss the model structure or hyperparameter setting in your design. (5%) (e.g. hyperparameters of transformer: d_model, nhead, d_hid , nlayers, dropout, etc. Why do you choose this setting?)
3. Kaggle Challenger Award! After Kaggle competition, the private leaderboard will reflect the final standings. In private leaderboard, you will be finally recognized as
   - Challenger (10%): Pass the Advanced Line
   - Second prize (5%): Pass the Baseline
   - Consolation prize (5%): Join Kaggle competition and pass Random Line

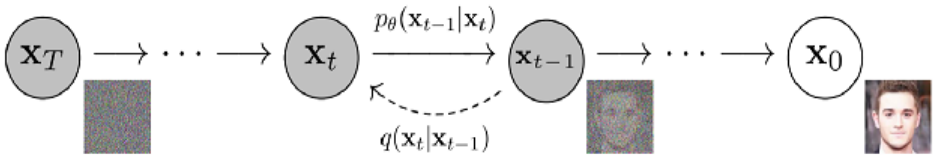
Hint

You might want to follow these steps to start your work:

1. Tokenize the given text data with some off-the-shelf software.
2. Build the vocabulary (like the dictionary object in python) to map the token into some unique ID.
3. Select the pretrained embedding (Glove, Fasttext, Word2vec) as the initialization of your embedding layer. (Not necessary, but recommended)
4. Construct your transformer model and finally end up with some simple feed forward module.
5. Choose the suitable optimizer (Adam might be not suitable) and activation function (ReLU might be not suitable. Try Tanh or Swish?)
6. Try some tricks like learning rate scheduler.
7. Check the given tutorial.

2 Image Generation (25%)

In this exercise, you will implement a Denoising Diffusion Probabilistic Model (DDPM) [1] to generate images by the provided **CelebFaces Attributes (CelebA) Dataset**. The Figure below shows the process of diffusion model where the model consists of a forward process which gradually adds noise, and the reverse process which transforms the noise back into a sample from the target distribution. Here are the link 1 and link 2 to the detailed introduction to diffusion model. Construct the DDPM by fulfilling the 2 TODOs. Notice that you are not



allowed to directly call the library or API to load the model. (20 %)

1. **Draw** some generated samples based on diffusion steps $T = 500$ and $T = 1000$. We will provide the **pre-trained weights** which are trained with 500 and 1000 steps. Hint: In the paper, the steps start at 1.
2. **Discuss** the results based on different diffusion steps. (5%)

3 Instruction-Tuning (25%)

In this homework assignment, using the instruction response dataset, you will fine-tune a pre-trained large language model, Llama 3 8B version. This task involves applying transfer learning to improve the model's performance on question-answering tasks.

Files Included:

- `train.json`
  1. idx: A unique identifier for the training record.
  2. input: This field provides additional context or data related to the cocktail recipe.
  3. instruction: A specific instruction or request related to generating a cocktail recipe.
  4. output: The expected output, which is the resulting cocktail recipe that meets the instruction and input requirements.
- `test.json`
  1. idx: A unique identifier for the testing record.
  2. input: This field provides additional context or data related to the cocktail recipe.
  3. instruction: A specific instruction or request related to generating a cocktail recipe.
- `sample_submission.json`
  1. idx: The unique identifier from `test.json`.
  2. output: This is where you will write the generated cocktail recipe based on the corresponding test record.

The generated output will be scored with BLEU score. The BLEU score is reported as a percentage, with higher values indicating better similarity between the generated text and the reference text.

Hint: we suggest using Google Colab to fine-tune llama3

Here are some resources that you can know more about

- unsloth
- Meta Llama 3

4 Rule

- Homework has two problems which can be separate into three parts:
  - **HW 2-1 is for Intent classification**:
    Please submit one zip file named **hw2_1_<StudentID>.zip** contains
    * hw2_1_<StudentID>.ipynb
    * submission.csv

    You can refer to expected_result_of_hw2-2.zip on E3 for more easily understanding.
  - **HW 2-2 is for Image generation**:
    Please submit **hw2_2_<StudentID>.ipynb** file which needs to contain all the results, codes, and reports for each exercise (e.g. **hw2_2_0123456.ipynb**).
  - **HW 2-3 is for Instruction-tuning**:
    Please submit one zip file named **hw2_3_<StudentID>.zip** contains
    * hw2_3_<StudentID>.ipynb
    * submission.json
- Implementation will be graded by
  - Completeness
  - Algorithm correctness
  - Discussion and analysis
- Only Python implementation is acceptable.
- Kindly suggest using GPU to run the code! You can refer to the Colab tutorial in E3.

References

[1] Jonathan Ho, Ajay Jain, and Pieter Abbeel, "Denoising diffusion probabilistic models," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.